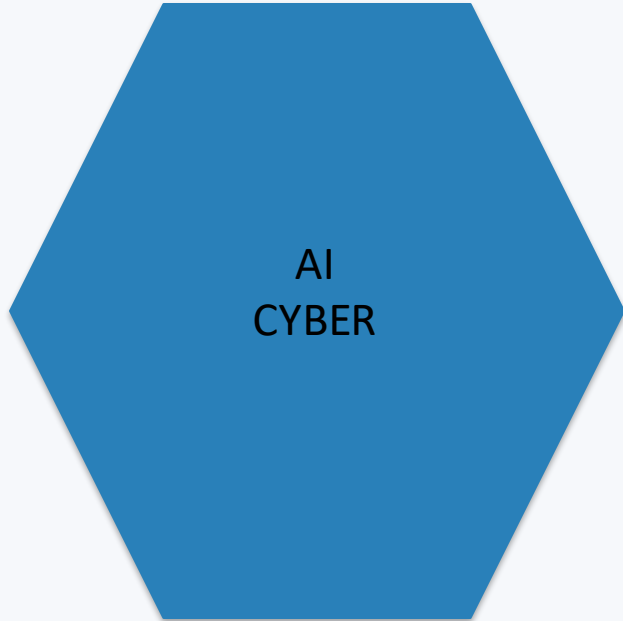


A Machine Learning System for Detecting Suspicious Login Attempts



- Course: Introduction to AI-Enhanced Cybersecurity
- Topic: Anomaly Detection in Login Logs
- Model: Isolation Forest
- Deliverables: Python code, dataset, results, README, presentation, and poster

The Cybersecurity Problem

Why do we need anomaly detection in logs?

- Attackers try to guess passwords or use leaked credentials.
- Login logs often contain early warning signs: many failed attempts, many usernames from the same IP, and activity at unusual hours.
- Manually reviewing thousands of events is inefficient, so an automated mechanism is needed.
- Goal: flag suspicious events for fast investigation by the security team.

Solution Idea

Learning normal behavior and flagging anomalies

1. Data Collection / Generation
Create a dataset of normal and suspicious login events.

2. Feature Extraction
Failed attempts, login hour, IP risk, attempt rate, and more.

3. Model Training
Isolation Forest learns the normal pattern and detects anomalies.

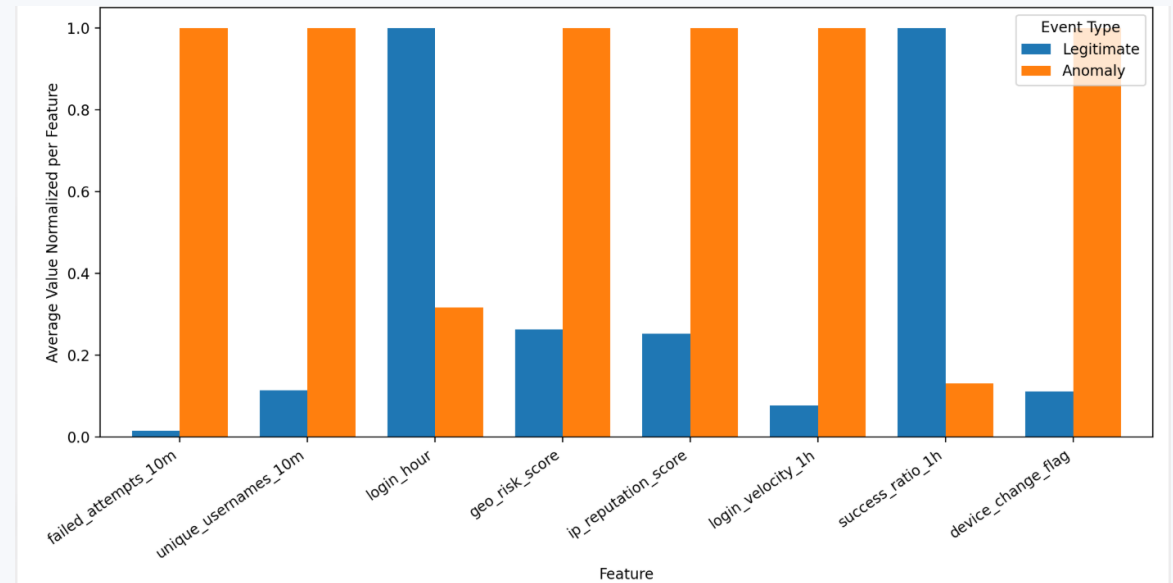
4. Alerting
Events with a high anomaly score are presented as suspicious.

Data Structure

The main features used in the system

- failed_attempts_10m - number of failed attempts in ten minutes
- unique_usernames_10m - number of different usernames from the same source
- login_hour - login hour
- geo_risk_score - geographic risk
- ip_reputation_score - IP reputation risk score
- login_velocity_1h - attempt rate per hour
- success_ratio_1h - success ratio
- device_change_flag - new device indicator

Feature Patterns



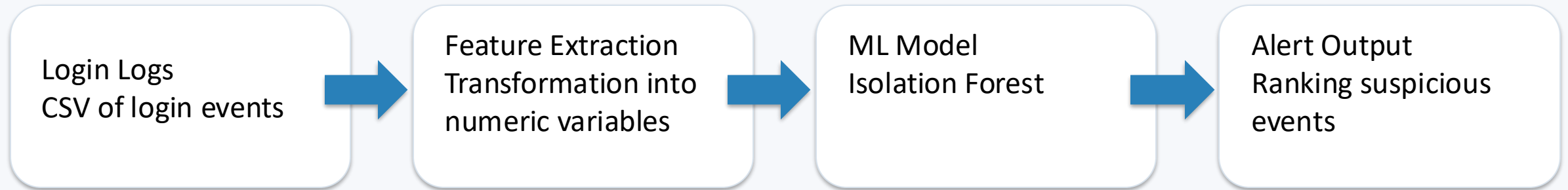
The Model: Isolation Forest

A simple and suitable algorithm for anomaly detection

- The model is trained mainly on normal examples.
- An event that is statistically easy to isolate is considered anomalous.
- It does not require a large number of real attack samples.
- The model returns a suspicion score for each event.
- It is especially suitable for an academic project that demonstrates cybersecurity + AI.

System Architecture

What happens from log entry to alert



Experiment Results

Performance metrics on the test set

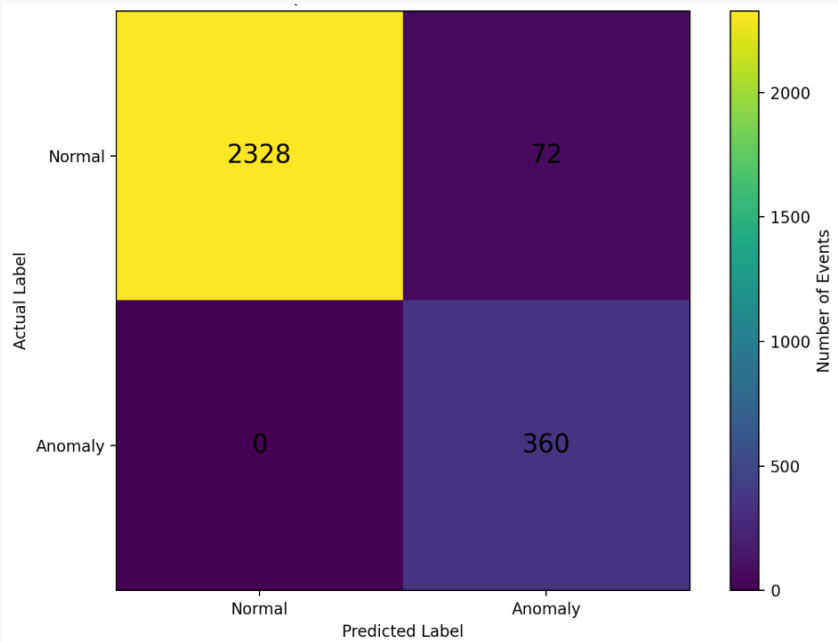
Accuracy
0.974

Precision
0.833

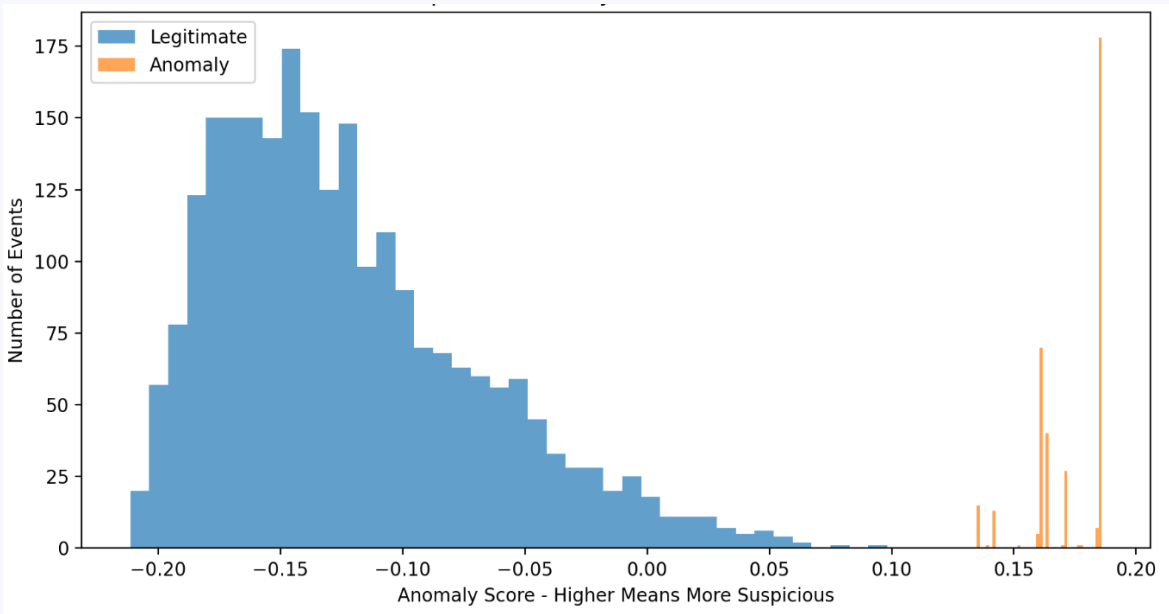
Recall
1.000

F1 Score
0.909

Confusion Matrix



Anomaly Score Distribution



Usage Example

Running the system from the command line

```
cd LogShield-AI/source_code  
pip install -r requirements.txt  
python generate_dataset.py  
python train_model.py  
python evaluate_model.py  
python main.py --failed 20 --users 3 --hour 2 --geo-risk 80 --ip-reputation 90 --velocity 100 --success-ratio 0.05 --device-change 1
```

- The output shows whether the event is normal or suspicious.
- The system also exports a file with the most suspicious events for investigation.

Limitations and Future Work

What can be improved in a future project

- The project uses synthetic data, so it does not replace real-world logs.
- The system can be connected to a SIEM or to an API that receives logs in real time.
- Additional features can be added, such as country, browser type, User-Agent, and source system.
- A dashboard can be built to display alerts, suspicion scores, and a short explanation for each event.

Summary

LogShield AI demonstrates the integration of cybersecurity and AI

- The system detects unusual login patterns based on log data.
- The selected model is simple, explainable, and suitable for anomaly detection.
- The deliverables include code, data, results, a README, a presentation, and a poster.
- The project can be run locally and demonstrates a cybersecurity scenario that is relevant to the real world.

Presentation Video: <https://youtu.be/nvUZAIELOYw>